Lab 19 – Code Translation: Converting Between Programming Languages

Task 1: Python to Java Conversion

Prompt:

I have a Python script that opens a file, reads its content line by line, counts the number of lines, and writes the count to another file.Can you help me convert this entire Python code into Java, making sure to use proper file streams like FileReader and FileWriter.Please include exception handling for file not found or IO errors, and ensure the output format matches exactly. Also, add console output to show the line count on the screen, just like the Python version does.Finally, test the Java code with a sample input file to confirm it produces the same results as the Python script.

Code:

```
def file_operations():
    try:
        with open('input.txt', 'r') as infile:
            lines = infile.readlines()
            line_count = len(lines)
        with open('output.txt', 'w') as outfile:
            outfile.write(f"Total lines: {line_count}\n")
        print(f"Total lines: {line_count}")
    except FileNotFoundError:
        print("Error: Input file not found.")
    except IOError:
        print("Error: IO operation failed.")

file_operations()
```

Output:

```java
// Converted Java code
import java.io.*;

public class FileOperations {
    public static void main(String[] args) {
        try {
            BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
            int lineCount = 0;
            while (reader.readLine() != null) {
                lineCount++;
            }
            reader.close();

            BufferedWriter writer = new BufferedWriter(new FileWriter("output.txt"));
            writer.write("Total lines: " + lineCount + "\n");
            writer.close();

            System.out.println("Total lines: " + lineCount);
        } catch (FileNotFoundException e) {
            System.out.println("Error: Input file not found.");
        } catch (IOException e) {
            System.out.println("Error: IO operation failed.");
        }
    }
}
```

Explanation:

The Java code uses BufferedReader and BufferedWriter for efficient file reading and writing.It counts lines by reading each line in a loop, similar to Python's readlines method.Exception handling catches file not found and IO errors, matching Python's try-except.The output format and console print are identical to ensure same behavior.

Task 2: C++ to Python Function Conversion

Prompt:

I have a C++ function that implements bubble sort on an array of integers.Can you convert this C++ sorting code into Python, keeping the same logic for swapping elements.Make sure the Python version sorts the list in ascending order and handles input correctly.Test it with different arrays like [3,1,4,1,5] and [9,8,7,6] to verify it works.Ensure the output shows the sorted array, just like the C++ version prints it.

Code:

```cpp
// Original C++ code
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
            }
        }
    }
}
```

Output:

```python
# Converted Python code
def bubble_sort(arr):
    n = len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
sorted_arr = bubble_sort(arr)
print("Sorted array:", " ".join(map(str, sorted_arr)))
```

Explanation:

The Python code uses a list instead of an array and swaps elements using tuple unpacking.It maintains the same nested loop structure for bubble sort comparison and swapping.The function returns the sorted list, and printing uses join for space-separated output.Testing with multiple arrays confirms it sorts correctly in ascending order.

Task 3: SQL to Pandas Query

Prompt:

I have a SQL query that groups sales data by department and calculates total sales and average price.Can you convert this SQL into Pandas code using groupby and agg functions on a DataFrame.Make sure to load the data from a CSV file and display the results in a table format.Test it with sample data having columns like department, sales, and price.Verify that the Pandas output matches the SQL grouped results exactly.

Code:

```sql
-- Original SQL query
SELECT department, SUM(sales) AS total_sales, AVG(price) AS avg_price
FROM sales_data
GROUP BY department;
```

Output:

```
# Converted Pandas code
import pandas as pd

# Load data
df = pd.read_csv('sales_data.csv')

# Group by department and aggregate
result = df.groupby('department').agg(
    total_sales=('sales', 'sum'),
    avg_price=('price', 'mean')
).reset_index()

print(result)
```

Explanation:

Pandas uses groupby on the department column to group rows together.The agg function applies sum to sales and mean to price, matching SQL aggregates.Reset_index makes department a regular column for better table display.The output table shows grouped totals and averages just like SQL results.

Task 4: Real-Time Application: Multi-Language Snippet Converter

Prompt:

I have a Python Flask app that creates a simple REST API endpoint to return a greeting message.Can you convert this Flask code into Java using Spring Boot with similar routing and response.Ensure the Java version runs on a server and returns JSON, like the Python version does.Test both by making HTTP requests and compare the responses they give.Document any differences in setup, libraries, and how the servers start up.

Code:

```python
# Original Python Flask code
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/greet')
def greet():
    return jsonify(message="Hello, World!")

if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```java
// Converted Java Spring Boot code
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.Map;
import java.util.HashMap;

@SpringBootApplication
@RestController
public class GreetingApp {
    @GetMapping("/greet")
    public Map<String, String> greet() {
        Map<String, String> response = new HashMap<>();
        response.put("message", "Hello, World!");
        return response;
    }

    public static void main(String[] args) {
        SpringApplication.run(GreetingApp.class, args);
    }
}
```

Explanation:

Java uses Spring Boot annotations for routing, unlike Flask's decorators. The response is a Map converted to JSON automatically by Spring. Setup requires Maven or Gradle for dependencies, while Python needs pip forFlask.Execution differs: Python runs with python app.py, Java with mvn spring-boot:run.

Task 5: Basic Function Translation (Python to C++)

Prompt:

I have a Python function that takes a number and checks if it's even or odd by using modulo.Can you convert this simple Python code into C++, with proper input from user and output print.Make sure to handle integer input and print "Even" or "Odd" based on the check.Test the C++ program with numbers like 4, 7, and 10 to ensure it works correctly.Compile and run it to confirm the output matches the Python version exactly.

Code:

```python
# Original Python code
def check_even_odd(num):
    if num % 2 == 0:
        print("Even")
    else:
        print("Odd")

# Example
check_even_odd(4)
```

Output:

```cpp
// Converted C++ code
#include <iostream>
using namespace std;

void check_even_odd(int num) {
    if (num % 2 == 0) {
        cout << "Even" << endl;
    } else {
        cout << "Odd" << endl;
    }
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    check_even_odd(num);
    return 0;
}
```

Explanation:

C++ uses cout for output and cin for input, unlike Python's print and input. The modulo operation num % 2 works the same way in both languages. The function structure is similar, but C++ requires main function and includes.Testing with multiple numbers confirms identical even/odd detection.